ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №8

«Мережеве та асинхронне програмування. REST API»

з курсу «Професійний Python»
Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 28.10.26

Львів — 2026

1. Мета роботи

Ознайомитися з основами мережевого програмування у Python, протоколом HTTP, принципами архітектури REST та форматом JSON. Навчитися розробляти REST API за допомогою FastAPI, валідувати структури даних через Pydantic v2, застосовувати асинхронні конструкції async/await, asyncio.Task та asyncio.gather() для конкурентного виконання операцій, обмежувати паралелізм за допомогою Semaphore, встановлювати timeout і реалізовувати retry з експоненційним backoff. Опанувати бібліотеку HTTPX (httpx.AsyncClient) та інтегрувати API з попередніми шарами репозиторіїв, сервісів і бази даних SQLite.

2. Індивідуальний варіант і постановка задачі

Варіант №3 — «Інтернет-магазин».

Ресурси: products (товари), categories (категорії товарів).

Основні операції: додавання товарів і категорій, пошук товарів за підрядком, фільтрація за категорією та діапазоном цін, часткове оновлення інформації про товар (PATCH), видалення (DELETE), транзакційна покупка товару (POST /products/{id}/purchase) зі списанням кількості та перевіркою залишку, розрахунок загальної вартості залишків на складі та визначення найдорожчого товару.

Асинхронне завдання: реалізувати конкурентне вивантаження інформації про декілька товарів через httpx.AsyncClient, об'єднати виконання через asyncio.Task та gather(), обмежити одночасні запити семафором (Semaphore=3), додати таймаут (timeout=5.0s) та механізм повторних спроб (retry).

3. Структура проєкту

eshop/

```
├── pyproject.toml              # Метадані, залежності та налаштування інструментів
├── README.md                   # Опис, інструкція встановлення, запуск та ендпоінти
├── report/
│   └── Вересюк_Ірина_Лаб08_V3.pdf    # Звіт з лабораторної роботи
├── src/
│   └── eshop/
│       ├── __init__.py         # Ініціалізація пакета
│       ├── database.py         # Підключення до БД (SQLAlchemy Engine, SessionLocal)
│       ├── models.py           # ORM-моделі бази даних (Category, Product)
│       ├── schemas.py          # Pydantic-схеми для валідації запитів та відповідей
│       ├── repositories.py     # Рівень доступу до даних (ProductRepository, CategoryRepository)
│       ├── services.py         # Сервісний шар (покупка, підрахунок статистики)
│       ├── external_api.py     # Асинхронний HTTP-клієнт (Task, gather, Semaphore, retry)
│       └── api.py              # Маршрутизація FastAPI (REST API ендпоінти)
```

└── tests/

  ├── conftest.py # Фікстури pytest, in-memory SQLite (StaticPool)

  └── test_api.py # Автоматизовані тести API та асинхронності

4. Опис створених модулів та повні лістинги коду

4.1. src/eshop/database.py

Модуль налаштовує підключення до реляційної бази даних SQLite за допомогою SQLAlchemy 2.0, створює рушій (engine), фабрику сесій SessionLocal та генератор залежностей get_session() для FastAPI через yield.

```
from collections.abc import Generator

from sqlalchemy import create_engine

from sqlalchemy.orm import declarative_base, sessionmaker, Session


DATABASE_URL = "sqlite:///./eshop.db"


engine = create_engine(

  DATABASE_URL,

  connect_args={"check_same_thread": False}

)


SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)


Base = declarative_base()


def get_session() -> Generator[Session, None, None]:

  session = SessionLocal()

  try:

    yield session

  finally:

    session.close()


def create_db_and_tables() -> None:

  Base.metadata.create_all(bind=engine)
```

4.2. src/eshop/models.py

Визначає декларативні моделі бази даних Category та Product. Забезпечує реляційний зв'язок один-до-багатьох (relationship) із зовнішнім ключем ForeignKey("categories.id") та каскадним видаленням cascade="all, delete-orphan".

```
from sqlalchemy import Column, Integer, String, Float, ForeignKey

from sqlalchemy.orm import relationship

from eshop.database import Base


class Category(Base):
  __tablename__ = "categories"


  id = Column(Integer, primary_key=True, index=True, autoincrement=True)
  name = Column(String(100), unique=True, nullable=False, index=True)
  description = Column(String(255), nullable=True)


  products = relationship(
    "Product",
    back_populates="category",
    cascade="all, delete-orphan"
  )


class Product(Base):
  __tablename__ = "products"


  id = Column(Integer, primary_key=True, index=True, autoincrement=True)
  name = Column(String(150), nullable=False, index=True)
  description = Column(String(500), nullable=True)
  price = Column(Float, nullable=False)
  stock = Column(Integer, nullable=False, default=0)
  category_id = Column(
    Integer,
    ForeignKey("categories.id", ondelete="CASCADE"),
    nullable=False
```

```
    )

    category = relationship("Category", back_populates="products")
```

4.3. src/eshop/schemas.py

Модуль містить Pydantic v2 моделі для семантичної валідації вхідних JSON-запитів (перевірка price > 0, stock >= 0, quantity > 0) та серіалізації вихідних даних через ConfigDict(from_attributes=True).

```
from pydantic import BaseModel, Field, ConfigDict


# --- Category Schemas ---
class CategoryBase(BaseModel):
    name: str = Field(min_length=2, max_length=100, description="Назва категорії")
    description: str | None = Field(default=None, max_length=255, description="Опис категорії")


class CategoryCreate(CategoryBase):
    pass


class CategoryResponse(CategoryBase):
    id: int
    model_config = ConfigDict(from_attributes=True)


# --- Product Schemas ---
class ProductBase(BaseModel):
    name: str = Field(min_length=2, max_length=150, description="Назва товару")
    description: str | None = Field(default=None, max_length=500, description="Опис товару")
    price: float = Field(gt=0, description="Ціна товару повинна бути строго більшою за 0")
    stock: int = Field(ge=0, description="Кількість на складі (>= 0)")
    category_id: int = Field(gt=0, description="Ідентифікатор категорії")


class ProductCreate(ProductBase):
    pass


class ProductUpdate(BaseModel):
```

```
    name: str | None = Field(default=None, min_length=2, max_length=150)
    description: str | None = Field(default=None, max_length=500)
    price: float | None = Field(default=None, gt=0)
    stock: int | None = Field(default=None, ge=0)
    category_id: int | None = Field(default=None, gt=0)

class ProductResponse(ProductBase):
    id: int
    model_config = ConfigDict(from_attributes=True)

# --- Purchase & Stats Schemas ---
class PurchaseRequest(BaseModel):
    quantity: int = Field(gt=0, description="Кількість товару для покупки")

class PurchaseResponse(BaseModel):
    product_id: int
    product_name: str
    quantity: int
    unit_price: float
    total_amount: float
    remaining_stock: int
    message: str

class InventoryStats(BaseModel):
    total_products_count: int
    total_items_in_stock: int
    total_inventory_value: float
    most_expensive_product: ProductResponse | None

class BatchProductRequest(BaseModel):
    product_ids: list[int] = Field(min_length=1, description="Список ID товарів")
```

4.4. src/eshop/repositories.py

Інкапсулює взаємодію з БД через SQLAlchemy Session. Класи CategoryRepository та ProductRepository реалізують CRUD-операції, повнотекстовий пошук за підрядком, діапазонні фільтри цін, підрахунок агрегованої вартості та пошук найдорожчого товару.

```
from sqlalchemy import select, func

from sqlalchemy.orm import Session

from eshop.models import Product, Category


class CategoryRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def get_all(self) -> list[Category]:
        stmt = select(Category).order_by(Category.name)
        return list(self.session.scalars(stmt))

    def get_by_id(self, category_id: int) -> Category | None:
        return self.session.get(Category, category_id)

    def get_by_name(self, name: str) -> Category | None:
        stmt = select(Category).where(Category.name == name)
        return self.session.scalars(stmt).first()

    def add(self, category: Category) -> Category:
        self.session.add(category)
        self.session.commit()
        self.session.refresh(category)
        return category

    def delete(self, category: Category) -> None:
        self.session.delete(category)
        self.session.commit()
```

```
class ProductRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def get_all(
        self,
        query: str | None = None,
        category_id: int | None = None,
        min_price: float | None = None,
        max_price: float | None = None,
        skip: int = 0,
        limit: int = 100,
    ) -> list[Product]:
        stmt = select(Product)
        if query:
            stmt = stmt.where(Product.name.ilike(f"%{query}%"))
        if category_id is not None:
            stmt = stmt.where(Product.category_id == category_id)
        if min_price is not None:
            stmt = stmt.where(Product.price >= min_price)
        if max_price is not None:
            stmt = stmt.where(Product.price <= max_price)
        stmt = stmt.offset(skip).limit(limit)
        return list(self.session.scalars(stmt))

    def get_by_id(self, product_id: int) -> Product | None:
        return self.session.get(Product, product_id)

    def get_by_category_id(self, category_id: int) -> list[Product]:
        stmt = select(Product).where(Product.category_id == category_id).order_by(Product.name)
        return list(self.session.scalars(stmt))
```

```
def add(self, product: Product) -> Product:
    self.session.add(product)
    self.session.commit()
    self.session.refresh(product)
    return product

def update(self, product: Product) -> Product:
    self.session.commit()
    self.session.refresh(product)
    return product

def delete(self, product: Product) -> None:
    self.session.delete(product)
    self.session.commit()

def get_most_expensive(self) -> Product | None:
    stmt = select(Product).order_by(Product.price.desc()).limit(1)
    return self.session.scalars(stmt).first()

def get_total_inventory_value(self) -> float:
    stmt = select(func.coalesce(func.sum(Product.price * Product.stock), 0.0))
    val = self.session.scalar(stmt)
    return float(val) if val is not None else 0.0

def get_total_stock_count(self) -> int:
    stmt = select(func.coalesce(func.sum(Product.stock), 0))
    val = self.session.scalar(stmt)
    return int(val) if val is not None else 0

def get_products_count(self) -> int:
    stmt = select(func.count(Product.id))
    val = self.session.scalar(stmt)
```

```
    return int(val) if val is not None else 0
```

4.5. src/eshop/services.py

Модуль бізнес-логіки InventoryService. Контролює правила предметної області: перевірку існування категорій, унікальність імен, часткове оновлення полів і транзакційну покупку товару зі списанням кількості та захистом від від'ємного залишку.

```
from sqlalchemy.orm import Session

from eshop.models import Product, Category

from eshop.repositories import ProductRepository, CategoryRepository

from eshop.schemas import (

  ProductCreate,

  ProductUpdate,

  CategoryCreate,

  PurchaseResponse,

  InventoryStats,

  ProductResponse,

)


class InventoryService:

  def __init__(self, session: Session) -> None:

    self.session = session

    self.product_repo = ProductRepository(session)

    self.category_repo = CategoryRepository(session)


  def create_category(self, data: CategoryCreate) -> Category:

    existing = self.category_repo.get_by_name(data.name)

    if existing:

      raise ValueError(f"Category '{data.name}' already exists")

    cat = Category(name=data.name, description=data.description)

    return self.category_repo.add(cat)


  def get_all_categories(self) -> list[Category]:

    return self.category_repo.get_all()
```

```
def get_category_by_id(self, category_id: int) -> Category | None:
    return self.category_repo.get_by_id(category_id)

def create_product(self, data: ProductCreate) -> Product:
    category = self.category_repo.get_by_id(data.category_id)
    if not category:
        raise ValueError(f"Category with ID {data.category_id} not found")
    prod = Product(
        name=data.name,
        description=data.description,
        price=data.price,
        stock=data.stock,
        category_id=data.category_id,
    )
    return self.product_repo.add(prod)

def get_product_by_id(self, product_id: int) -> Product | None:
    return self.product_repo.get_by_id(product_id)

def search_products(
    self,
    query: str | None = None,
    category_id: int | None = None,
    min_price: float | None = None,
    max_price: float | None = None,
    skip: int = 0,
    limit: int = 100,
) -> list[Product]:
    return self.product_repo.get_all(
        query=query,
        category_id=category_id,
        min_price=min_price,
```

```
            max_price=max_price,
            skip=skip,
            limit=limit,
        )

    def get_products_by_category(self, category_id: int) -> list[Product]:
        if not self.category_repo.get_by_id(category_id):
            raise KeyError(f"Category with ID {category_id} not found")
        return self.product_repo.get_by_category_id(category_id)

    def update_product(self, product_id: int, data: ProductUpdate) -> Product | None:
        product = self.product_repo.get_by_id(product_id)
        if not product:
            return None

        if data.category_id is not None:
            cat = self.category_repo.get_by_id(data.category_id)
            if not cat:
                raise ValueError(f"Category with ID {data.category_id} not found")

        updates = data.model_dump(exclude_unset=True)
        for field, value in updates.items():
            setattr(product, field, value)

        return self.product_repo.update(product)

    def delete_product(self, product_id: int) -> bool:
        product = self.product_repo.get_by_id(product_id)
        if not product:
            return False
        self.product_repo.delete(product)
        return True
```

```
def purchase_product(self, product_id: int, quantity: int) -> PurchaseResponse:
    product = self.product_repo.get_by_id(product_id)
    if not product:
        raise KeyError(f"Product with ID {product_id} not found")

    if product.stock < quantity:
        raise ValueError(
            f"Insufficient stock for '{product.name}'. Requested: {quantity}, Available: {product.stock}"
        )

    # Транзакційне списання кількості
    product.stock -= quantity
    total = round(product.price * quantity, 2)
    self.session.commit()
    self.session.refresh(product)

    return PurchaseResponse(
        product_id=product.id,
        product_name=product.name,
        quantity=quantity,
        unit_price=product.price,
        total_amount=total,
        remaining_stock=product.stock,
        message="Purchase successful",
    )

def get_inventory_stats(self) -> InventoryStats:
    count = self.product_repo.get_products_count()
    total_items = self.product_repo.get_total_stock_count()
    val = round(self.product_repo.get_total_inventory_value(), 2)
    most_expensive = self.product_repo.get_most_expensive()
```

```
    most_exp_resp = (
        ProductResponse.model_validate(most_expensive) if most_expensive else None
    )

    return InventoryStats(
        total_products_count=count,
        total_items_in_stock=total_items,
        total_inventory_value=val,
        most_expensive_product=most_exp_resp,
    )
```

4.6. src/eshop/external_api.py

Реалізує вимоги асинхронності: застосування клієнта httpx.AsyncClient, обгортання корутин у asyncio.create_task(), конкурентне очікування через asyncio.gather(), обмеження паралелізму семафором asyncio.Semaphore, таймаут wait_for та повторні спроби retry з експоненційним backoff.

```
import asyncio
import logging
import httpx

logger = logging.getLogger(__name__)

async def fetch_product_with_retry(
    client: httpx.AsyncClient,
    semaphore: asyncio.Semaphore,
    base_url: str,
    product_id: int,
    max_retries: int = 3,
) -> dict:
    url = f"{base_url.rstrip('/')}/products/{product_id}"
    async with semaphore:
        for attempt in range(1, max_retries + 1):
            try:
```

```
            response = await client.get(url)
            response.raise_for_status()
            return response.json()
        except (httpx.RequestError, httpx.HTTPStatusError) as exc:
            logger.warning(
                f"Спроба {attempt}/{max_retries} для {url} завершилась помилкою: {exc}"
            )
            if attempt == max_retries:
                return {
                    "id": product_id,
                    "error": str(exc),
                    "status": "failed",
                }
            # Експоненційний backoff
            await asyncio.sleep(0.1 * (2 ** (attempt - 1)))

    return {"id": product_id, "error": "Unknown error", "status": "failed"}


async def fetch_multiple_products_async(
    base_url: str,
    product_ids: list[int],
    max_concurrency: int = 3,
    timeout_seconds: float = 5.0,
    client: httpx.AsyncClient | None = None,
) -> list[dict]:
    semaphore = asyncio.Semaphore(max_concurrency)

    async def _run(c: httpx.AsyncClient) -> list[dict]:
        tasks = [
            asyncio.create_task(
                fetch_product_with_retry(
                    client=c,
```

```
            semaphore=semaphore,
            base_url=base_url,
            product_id=pid,
            max_retries=3,
          )
        )
        for pid in product_ids
    ]
    return list(await asyncio.gather(*tasks))


  try:
    if client is not None:
      return await asyncio.wait_for(_run(client), timeout=timeout_seconds)
    else:
      async with httpx.AsyncClient(timeout=timeout_seconds) as new_client:
        return await asyncio.wait_for(_run(new_client), timeout=timeout_seconds)
  except asyncio.TimeoutError:
    logger.error(f"Асинхронний запит перевищив ліміт часу {timeout_seconds}с")
    raise
```

4.7. src/eshop/api.py

Головний модуль FastAPI. Реєструє маршрутизатори для категорій, товарів, транзакційної покупки, статистики та асинхронного пакетного завантаження. Забезпечує правильні HTTP-статуси (200, 201, 204, 400, 404, 409, 504) та створення таблиць БД через lifespan.

```
import asyncio
from contextlib import asynccontextmanager
from typing import AsyncGenerator
from fastapi import FastAPI, Depends, HTTPException, Query, status
from sqlalchemy.orm import Session

from eshop.database import get_session, create_db_and_tables
from eshop.models import Product, Category
from eshop.schemas import (
  ProductCreate,
```

```
    ProductUpdate,
    ProductResponse,
    CategoryCreate,
    CategoryResponse,
    PurchaseRequest,
    PurchaseResponse,
    InventoryStats,
    BatchProductRequest,
)
from eshop.services import InventoryService
from eshop.external_api import fetch_multiple_products_async


@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncGenerator[None, None]:
    create_db_and_tables()
    yield

app = FastAPI(
    title="E-Shop REST API",
    description="Інтернет-магазин (Лабораторна робота №8, Варіант 3) — Вересюк Ірина",
    version="1.0.0",
    lifespan=lifespan,
)


@app.get("/", tags=["System"])
async def root() -> dict[str, str]:
    return {
        "status": "online",
        "service": "E-Shop REST API",
        "version": "1.0.0",
        "documentation": "/docs",
    }
```

```
# --- Categories ---
@app.post(
    "/categories",
    response_model=CategoryResponse,
    status_code=status.HTTP_201_CREATED,
    tags=["Categories"],
)
async def create_category(
    data: CategoryCreate,
    session: Session = Depends(get_session),
) -> Category:
    service = InventoryService(session)
    try:
        return service.create_category(data)
    except ValueError as e:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(e))


@app.get(
    "/categories",
    response_model=list[CategoryResponse],
    tags=["Categories"],
)
async def list_categories(
    session: Session = Depends(get_session),
) -> list[Category]:
    service = InventoryService(session)
    return service.get_all_categories()


@app.get(
    "/categories/{category_id}/products",
    response_model=list[ProductResponse],
```

```
    tags=["Categories", "Products"],
)
async def get_products_by_category(
    category_id: int,
    session: Session = Depends(get_session),
) -> list[Product]:
    service = InventoryService(session)
    try:
        return service.get_products_by_category(category_id)
    except KeyError as e:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(e))


# --- Products ---
@app.get(
    "/products",
    response_model=list[ProductResponse],
    tags=["Products"],
)
async def list_products(
    query: str | None = Query(default=None, description="Пошук за назвою"),
    category_id: int | None = Query(default=None, description="Фільтр за категорією"),
    min_price: float | None = Query(default=None, ge=0, description="Мінімальна ціна"),
    max_price: float | None = Query(default=None, ge=0, description="Максимальна ціна"),
    skip: int = Query(default=0, ge=0, description="Зсув пагінації"),
    limit: int = Query(default=100, ge=1, le=100, description="Кількість записів"),
    session: Session = Depends(get_session),
) -> list[Product]:
    service = InventoryService(session)
    return service.search_products(
        query=query,
        category_id=category_id,
        min_price=min_price,
```

```
        max_price=max_price,
        skip=skip,
        limit=limit,
    )

@app.get(
    "/products/{product_id}",
    response_model=ProductResponse,
    tags=["Products"],
)
async def get_product(
    product_id: int,
    session: Session = Depends(get_session),
) -> Product:
    service = InventoryService(session)
    prod = service.get_product_by_id(product_id)
    if not prod:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID {product_id} not found",
        )
    return prod

@app.post(
    "/products",
    response_model=ProductResponse,
    status_code=status.HTTP_201_CREATED,
    tags=["Products"],
)
async def create_product(
    data: ProductCreate,
    session: Session = Depends(get_session),
```

```
) -> Product:
    service = InventoryService(session)
    try:
        return service.create_product(data)
    except ValueError as e:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(e))


@app.patch(
    "/products/{product_id}",
    response_model=ProductResponse,
    tags=["Products"],
)
async def update_product(
    product_id: int,
    data: ProductUpdate,
    session: Session = Depends(get_session),
) -> Product:
    service = InventoryService(session)
    try:
        updated = service.update_product(product_id, data)
        if not updated:
            raise HTTPException(
                status_code=status.HTTP_404_NOT_FOUND,
                detail=f"Product with ID {product_id} not found",
            )
        return updated
    except ValueError as e:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=str(e))


@app.delete(
    "/products/{product_id}",
    status_code=status.HTTP_204_NO_CONTENT,
```

```
    tags=["Products"],
)
async def delete_product(
    product_id: int,
    session: Session = Depends(get_session),
) -> None:
    service = InventoryService(session)
    deleted = service.delete_product(product_id)
    if not deleted:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Product with ID {product_id} not found",
        )


@app.post(
    "/products/{product_id}/purchase",
    response_model=PurchaseResponse,
    tags=["Transactions"],
)
async def purchase_product(
    product_id: int,
    data: PurchaseRequest,
    session: Session = Depends(get_session),
) -> PurchaseResponse:
    service = InventoryService(session)
    try:
        return service.purchase_product(product_id, data.quantity)
    except KeyError as e:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(e))
    except ValueError as e:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(e))
```

```
# --- Statistics & Async Operations ---
@app.get(
    "/statistics/inventory",
    response_model=InventoryStats,
    tags=["Statistics"],
)
async def get_inventory_statistics(
    session: Session = Depends(get_session),
) -> InventoryStats:
    service = InventoryService(session)
    return service.get_inventory_stats()


@app.post(
    "/products/batch-fetch",
    tags=["Async Operations"],
)
async def batch_fetch_products(
    request: BatchProductRequest,
) -> list[dict]:
    try:
        return await fetch_multiple_products_async(
            base_url="http://127.0.0.1:8000",
            product_ids=request.product_ids,
            max_concurrency=3,
            timeout_seconds=5.0,
        )
    except asyncio.TimeoutError:
        raise HTTPException(
            status_code=status.HTTP_504_GATEWAY_TIMEOUT,
            detail="Batch request timed out",
        )
```

4.8. tests/conftest.py

Конфігурація середовища тестування. Використовує StaticPool для збереження стану in-memory бази SQLite між викликами сесій, перевизначає залежність get_session у FastAPI та надає фікстури client (TestClient) і async_client (httpx.AsyncClient).

```
import pytest

import pytest_asyncio

from sqlalchemy import create_engine

from sqlalchemy.orm import sessionmaker

from sqlalchemy.pool import StaticPool

from fastapi.testclient import TestClient

from httpx import AsyncClient, ASGITransport


from eshop.database import Base, get_session

from eshop.api import app

from eshop.models import Category, Product


# StaticPool гарантує, що in-memory SQLite база зберігає стан між сесіями в межах тестів

TEST_DATABASE_URL = "sqlite:///:memory:"


engine = create_engine(

  TEST_DATABASE_URL,

  connect_args={"check_same_thread": False},

  poolclass=StaticPool,

)

TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)


def override_get_session():

  session = TestingSessionLocal()

  try:

    yield session

  finally:

    session.close()
```

```
# Підміняємо залежність бази даних у FastAPI на тестову
app.dependency_overrides[get_session] = override_get_session

@pytest.fixture(autouse=True)
def setup_database():
    # Створюємо таблиці перед кожним тестом
    Base.metadata.create_all(bind=engine)
    session = TestingSessionLocal()

    # Початкові дані
    cat1 = Category(name="Ноутбуки", description="Лептопи та ультрабуки")
    cat2 = Category(name="Смартфони", description="Мобільні телефони")
    session.add_all([cat1, cat2])
    session.commit()

    prod1 = Product(name="MacBook Pro 16", description="M3 Max 36GB", price=120000.0, stock=5,
category_id=cat1.id)
    prod2 = Product(name="Asus ROG Zephyrus", description="Ryzen 9 RTX 4080", price=95000.0,
stock=8, category_id=cat1.id)
    prod3 = Product(name="iPhone 15 Pro", description="Titanium 256GB", price=48000.0, stock=15,
category_id=cat2.id)
    session.add_all([prod1, prod2, prod3])
    session.commit()
    session.close()

    yield

    # Очищаємо таблиці після завершення тесту
    Base.metadata.drop_all(bind=engine)

@pytest.fixture
def client():
    with TestClient(app) as c:
```

```
    yield c


@pytest_asyncio.fixture
async def async_client():
  transport = ASGITransport(app=app)
  async with AsyncClient(transport=transport, base_url="http://test") as ac:
    yield ac
```

4.9. tests/test_api.py

Набір з 14 автоматизованих тестів, що покривають перевірку статус-кодів, CRUD-операцій, Pydantic-валідації помилкових цін (422), конфліктів унікальності (409), транзакційної покупки та асинхронного клієнта**.**

```
import pytest
from httpx import AsyncClient
from eshop.external_api import fetch_multiple_products_async


def test_root(client):
  response = client.get("/")
  assert response.status_code == 200
  assert response.json()["status"] == "online"


def test_get_categories(client):
  response = client.get("/categories")
  assert response.status_code == 200
  assert len(response.json()) >= 2


def test_create_category(client):
  response = client.post("/categories", json={"name": "Планшети", "description": "Планшетні ПК"})
  assert response.status_code == 201
  assert response.json()["name"] == "Планшети"


def test_create_category_duplicate(client):
  response = client.post("/categories", json={"name": "Ноутбуки", "description": "Повтор"})
  assert response.status_code == 409
```

```
def test_list_products_and_search(client):
    response = client.get("/products")
    assert response.status_code == 200
    assert len(response.json()) == 3

    response_search = client.get("/products?query=MacBook")
    assert response_search.status_code == 200
    assert len(response_search.json()) == 1

def test_get_product_by_id(client):
    response = client.get("/products/1")
    assert response.status_code == 200
    assert response.json()["id"] == 1

    response_404 = client.get("/products/999")
    assert response_404.status_code == 404

def test_create_product(client):
    new_prod = {
        "name": "Lenovo Legion 5",
        "description": "Ігровий лептоп",
        "price": 54000.0,
        "stock": 10,
        "category_id": 1
    }
    response = client.post("/products", json=new_prod)
    assert response.status_code == 201
    assert response.json()["name"] == "Lenovo Legion 5"

def test_validation_invalid_price(client):
    invalid_prod = {
```

```
        "name": "Invalid Price Item",
        "description": "Test",
        "price": -100.0,
        "stock": 5,
        "category_id": 1
    }
    response = client.post("/products", json=invalid_prod)
    assert response.status_code == 422

def test_update_product_patch(client):
    update_data = {"price": 115000.0, "stock": 4}
    response = client.patch("/products/1", json=update_data)
    assert response.status_code == 200
    assert response.json()["price"] == 115000.0

def test_delete_product(client):
    response = client.delete("/products/1")
    assert response.status_code == 204
    assert client.get("/products/1").status_code == 404

def test_purchase_product_success(client):
    response = client.post("/products/2/purchase", json={"quantity": 3})
    assert response.status_code == 200
    data = response.json()
    assert data["quantity"] == 3
    assert data["remaining_stock"] == 5
    assert data["total_amount"] == 95000.0 * 3

def test_purchase_product_insufficient_stock(client):
    response = client.post("/products/2/purchase", json={"quantity": 100})
    assert response.status_code == 409
    assert "Insufficient stock" in response.json()["detail"]
```

```
def test_inventory_statistics(client):
    response = client.get("/statistics/inventory")
    assert response.status_code == 200
    data = response.json()
    assert data["total_products_count"] >= 1
    assert data["total_inventory_value"] > 0
    assert data["most_expensive_product"] is not None


@pytest.mark.asyncio
async def test_async_fetch_multiple_products(async_client: AsyncClient):
    results = await fetch_multiple_products_async(
        base_url="http://test",
        product_ids=[1, 2, 3],
        max_concurrency=2,
        timeout_seconds=5.0,
        client=async_client,
    )
    assert len(results) == 3
    ids = [r["id"] for r in results]
    assert 1 in ids and 2 in ids and 3 in ids
```

5. Перелік ендпоінтів REST API

Метод	**Шлях (Endpoint)**	**Призначення**	**Коди відповіді**
GET	**/**	**Перевірка стану сервісу (Health check)**	**200**
POST	**/categories**	**Створення нової категорії товарів**	**201, 409, 422**
GET	**/categories**	**Отримання списку всіх категорій**	**200**

Метод	Шлях (Endpoint)	Призначення	Коди відповіді
GET	/categories/{category_id}/products	Список товарів обраної категорії	200, 404
GET	/products	Пошук за назвою, фільтрація за ціною та пагінація	200
GET	/products/{product_id}	Отримання товару за ідентифікатором	200, 404
POST	/products	Створення нового товару	201, 400, 422
PATCH	/products/{product_id}	Часткове оновлення інформації про товар	200, 404
DELETE	/products/{product_id}	Видалення товару за ID	204, 404
POST	/products/{product_id}/purchase	Транзакційне списання кількості при покупці	200, 404, 409
GET	/statistics/inventory	Аналітика: загальна вартість та найдорожчий товар	200
POST	/products/batch-fetch	Конкурентне вивантаження (gather + Semaphore + retry)	200, 504

6. Приклади HTTP-запитів та відповідей JSON

6.1. Створення товару (POST /products)

POST /products HTTP/1.1

Content-Type: application/json

{

```
 "name": "Apple MacBook Air 15",
 "description": "M3 16GB 512GB",
 "price": 64000.0,
 "stock": 10,
 "category_id": 1
}
```


HTTP/1.1 201 Created

Content-Type: application/json


```
{
 "name": "Apple MacBook Air 15",
 "description": "M3 16GB 512GB",
 "price": 64000.0,
 "stock": 10,
 "category_id": 1,
 "id": 1
}
```


6.2. Покупка товару (POST /products/1/purchase)

POST /products/1/purchase HTTP/1.1

Content-Type: application/json


```
{
 "quantity": 2
}
```


HTTP/1.1 200 OK

Content-Type: application/json


```
{
 "product_id": 1,
 "product_name": "Apple MacBook Air 15",
```

```
 "quantity": 2,
 "unit_price": 64000.0,
 "total_amount": 128000.0,
 "remaining_stock": 8,
 "message": "Purchase successful"
}
```


6.3. Статистика складу (GET /statistics/inventory)

HTTP/1.1 200 OK

Content-Type: application/json


```
{
 "total_products_count": 1,
 "total_items_in_stock": 8,
 "total_inventory_value": 512000.0,
 "most_expensive_product": {
  "name": "Apple MacBook Air 15",
  "description": "M3 16GB 512GB",
  "price": 64000.0,
  "stock": 8,
  "category_id": 1,
  "id": 1
 }
}
```


7. Робота з Git

Усі файли проєкту зафіксовано в системі контролю версій Git за допомогою трьох логічних послідовних комітів.

7.1. Команди створення комітів

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)

$ git add pyproject.toml README.md .gitignore src/eshop/__init__.py src/eshop/database.py src/eshop/models.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)

$ git commit -m "Add project configuration, database setup, and ORM models"

[main (root-commit) a298d10] Add project configuration, database setup, and ORM models
 6 files changed, 127 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 pyproject.toml
 create mode 100644 src/eshop/__init__.py
 create mode 100644 src/eshop/database.py
 create mode 100644 src/eshop/models.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)
$ git add src/eshop/schemas.py src/eshop/repositories.py src/eshop/services.py src/eshop/api.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)
$ git commit -m "Implement REST API endpoints, schemas, and inventory services"
[main ed71072] Implement REST API endpoints, schemas, and inventory services
 4 files changed, 547 insertions(+)
 create mode 100644 src/eshop/api.py
 create mode 100644 src/eshop/repositories.py
 create mode 100644 src/eshop/schemas.py
 create mode 100644 src/eshop/services.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)
$ git add src/eshop/external_api.py tests/

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)
$ git commit -m "Add async client with retry/semaphore and test suite"
[main 5ade583] Add async client with retry/semaphore and test suite
 3 files changed, 262 insertions(+)
 create mode 100644 src/eshop/external_api.py
 create mode 100644 tests/conftest.py
 create mode 100644 tests/test_api.py

7.2. Результат виконання git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)

$ git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

7.3. Результат виконання git log –oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/eshop (main)

$ git log --oneline

5ade583 (HEAD -> main, origin/main) Add async client with retry/semaphore and test suite

ed71072 Implement REST API endpoints, schemas, and inventory services

a298d10 Add project configuration, database setup, and ORM models

8. Результати автоматизованого тестування (pytest)

Усі 14 тестів виконані за 0.48 секунди без жодної помилки.

(.venv) PS D:\Робочий стіл\проф. python\eshop> pytest -v

D:\Робочий стіл\проф. python\eshop\.venv\Lib\site-packages\pytest_asyncio\plugin.py:300: PytestDeprecationWarning: The configuration option "asyncio_default_fixture_loop_scope" is unset.

The event loop scope for asynchronous fixtures will default to the "fixture" caching scope. Future versions of pytest-asyncio will default the loop scope for asynchronous fixtures to "function" scope. Set the default fixture loop scope explicitly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module", "package", "session"

warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))

========== test session starts ==========

platform win32 -- Python 3.13.0, pytest-9.2.0, pluggy-1.7.0 -- D:\Робочий стіл\проф. python\eshop\.venv\Scripts\python.exe

cachedir: .pytest_cache

rootdir: D:\Робочий стіл\проф. python\eshop

configfile: pyproject.toml

plugins: anyio-4.15.1, asyncio-1.4.1

asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function

collected 14 items

tests/test_api.py::test_root PASSED [7%]

tests/test_api.py::test_get_categories PASSED [14%]

tests/test_api.py::test_create_category PASSED [21%]

tests/test_api.py::test_create_category_duplicate PASSED [28%]

tests/test_api.py::test_list_products_and_search PASSED [35%]

tests/test_api.py::test_get_product_by_id PASSED [42%]

tests/test_api.py::test_create_product PASSED [50%]

tests/test_api.py::test_validation_invalid_price PASSED [57%]

tests/test_api.py::test_update_product_patch PASSED [64%]

tests/test_api.py::test_delete_product PASSED [71%]

tests/test_api.py::test_purchase_product_success PASSED [78%]

tests/test_api.py::test_purchase_product_insufficient_stock PASSED [85%]

tests/test_api.py::test_inventory_statistics PASSED [92%]

tests/test_api.py::test_async_fetch_multiple_products PASSED [100%]

========== warnings summary ==========

.venv\Lib\site-packages\fastapi\testclient.py:1

D:\Робочий стіл\проф. python\eshop\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.

from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

========== 14 passed, 1 warning in 0.48s ==========

9. Висновок

У ході лабораторної роботи №8 розроблено асинхронний REST API для вебсервісу інтернет-магазину (Варіант №3) з використанням FastAPI, Pydantic v2 та SQLAlchemy 2.0. Реалізовано повний набір CRUD-операцій, транзакційну логіку списання товарів зі складу при покупці та обчислення аналітичних показників залишків. Створено асинхронний клієнт з використанням httpx.AsyncClient, asyncio.Task, asyncio.gather(), з лімітуванням навантаження через Semaphore, таймаутами та повторними спробами (retry). Забезпечено 100% проходження 14 інтеграційних тестів pytest та сформовано структуровану історію комітів у Git.